16-10-2026

Sistema de enrutamiento para tiendas de autoservicio

https://github.com/Pacheco17

DIEGO PACHECO PEREZ
IGNACIO RAMOS VILLA

1- Introducción

La distribución de productos de abarrotes en una red urbana como Culiacán, Sinaloa, representa un problema logístico de alta complejidad. La cadena de restaurantes cuenta con 10 centros de distribución (CEDIS) y 90 tiendas dispersas geográficamente, lo que genera un espacio de soluciones combinatorio enorme donde encontrar la ruta óptima por fuerza bruta es computacionalmente inviable. Ante esto, se recurre a métodos heurísticos que permiten encontrar soluciones de alta calidad en tiempo razonable.

Minimizar únicamente distancia podría llevar a rutas que, aunque cortas en km, resulten costosas si el consumo de combustible no es proporcional (por ejemplo, en tramos con pendiente, tráfico o tipo de vía). La función ponderada captura mejor la realidad operativa.

2- Objetivo

Crear una estrategia para mejorar las rutas de entrega de productos de abarrotes desde los centros de distribución hasta las sucursales de una cadena de restaurantes en Culiacán, Sinaloa. Para ello se utilizará el método heurístico de **Recocido Simulado**, con el objetivo de reducir costos de operación, hacer más eficiente la logística y encontrar rutas de entrega que sean muy cercanas a la mejor solución posible.

3- Objetivos específicos

- Preparar y organizar los datos relacionados con la distribución, como las sucursales, los centros de distribución, las distancias entre ellos y el costo del combustible, para representar correctamente el problema logístico.

- Crear una solución inicial de rutas de entrega y evaluarla, calculando distancias, costos y consumo de gasolina, de manera que sirva como punto de partida para el proceso de optimización.

- Utilizar el algoritmo de **Recocido Simulado** para ir generando nuevas rutas (vecinas) y mejorar las existentes de forma iterativa, buscando reducir costos, distancias recorridas y consumo de combustible.

4- Desarrollo

El presente proyecto analiza el reto logístico que enfrenta una cadena de restaurantes en Culiacán, Sinaloa, para distribuir productos de abarrotes desde sus centros de distribución hacia sus distintas sucursales. La red logística de la empresa está formada por 10 centros de distribución que abastecen a un total de 90 tiendas ubicadas en diferentes puntos de la ciudad.

Este escenario corresponde a un Problema de Ruteo de Vehículos (PRV), el cual consiste en diseñar rutas de entrega eficientes que permitan reducir costos operativos y mejorar el desempeño del sistema logístico (Laporte, 1992). La dificultad del problema se debe a la gran cantidad de combinaciones de rutas posibles, lo que hace poco práctico encontrar la solución óptima mediante una búsqueda completa en un tiempo razonable. Por esta razón, el objetivo principal es minimizar un costo total ponderado que toma en cuenta dos factores principales: la distancia total recorrida y el consumo de combustible relacionado con esas rutas.

Para abordar el problema, el alcance del proyecto se enfoca en la optimización estática de las rutas. El propósito es reducir una función de costo ponderado que, como se implementa en el programa, considera tanto la distancia total recorrida como el consumo de combustible. Debido a esto, el modelo no toma en cuenta variables externas dinámicas como el tráfico en tiempo real, la demanda particular de cada sucursal o la prioridad de ciertas entregas, concentrándose únicamente en encontrar la configuración de rutas más eficiente a partir de los datos geográficos y de costos disponibles.

- Explicación del recocido

Para abordar el problema de optimización de rutas de distribución planteado en este proyecto, se seleccionó el algoritmo de Recocido Simulado como estrategia de solución. Esta elección se fundamenta en diversos criterios que se detallan a continuación. El Recocido Simulado es un algoritmo probabilístico de optimización inspirado en el proceso de recocido en metalurgia, en el que un material se calienta y luego se enfría lentamente para modificar sus propiedades físicas. El algoritmo imita este proceso para buscar una aproximación al óptimo global de una función dada. Una de sus características distintivas es que, además de aceptar soluciones que mejoran la función, también puede aceptar soluciones ‘peores’ con cierta probabilidad, lo cual le permite escapar de óptimos locales

Considerando estas características, el Recocido Simulado se consolida como una estrategia idónea para este proyecto. Su capacidad para abordar la complejidad NPdifícil del problema, junto con su mecanismo probabilístico para evadir óptimos locales, lo posicionan como un método robusto. Adicionalmente, su probada efectividad, flexibilidad y relativa simplicidad de implementación justifican plenamente su selección como el enfoque de optimización a seguir.

- Implementación

Una vez definidas las herramientas para la gestión de datos y la visualización de resultados, el núcleo del algoritmo de Recocido Simulado se desarrolló de forma modular. El proceso se organizó mediante varias funciones conectadas entre sí, donde cada una cumple una tarea específica dentro de la lógica del programa.

En primer lugar, se implementaron funciones encargadas de cargar y preparar la información proveniente de los archivos de datos. Estas funciones se encargan de leer los archivos .csv, limpiar los datos cuando es necesario y convertirlos en estructuras adecuadas para su uso dentro del algoritmo.

Posteriormente, se definieron las funciones relacionadas con el cálculo de distancias y costos. A partir de las coordenadas geográficas de los centros de distribución y de las tiendas, el programa calcula las distancias entre puntos y estima el costo asociado considerando el consumo de combustible. Estos valores son fundamentales para evaluar la calidad de cada solución generada.

Después se desarrolló la función encargada de generar una solución inicial. Esta solución establece una asignación inicial de las tiendas a los centros de distribución y define una primera configuración de rutas, la cual sirve como punto de partida para el proceso de optimización.

A partir de esta solución inicial, el algoritmo comienza a generar soluciones vecinas mediante operaciones como el intercambio de nodos entre rutas o la reasignación de tiendas a diferentes centros de distribución. Cada nueva solución se evalúa utilizando la función de costo total, lo que permite determinar si representa una mejora respecto a la solución actual.

El algoritmo de Recocido Simulado utiliza además un mecanismo de aceptación probabilística, que permite aceptar temporalmente soluciones menos eficientes para evitar quedar atrapado en mínimos locales. A medida que avanzan las iteraciones, la temperatura del sistema disminuye gradualmente, reduciendo la probabilidad de aceptar soluciones peores y guiando el proceso hacia una solución más estable y eficiente.

Finalmente, una vez que el algoritmo completa sus iteraciones, se obtiene la mejor solución encontrada. Esta solución se analiza mediante gráficas que muestran la evolución del costo, la distancia recorrida y el consumo de combustible. Además, se genera un mapa interactivo que permite visualizar claramente las rutas finales entre los centros de distribución y las sucursales.

5- Implementación computacional y resultados

Solución Inicial

Para comenzar la optimización, necesitábamos un punto de partida que fuera estructuralmente válido y geográficamente razonable. Descartamos una solución aleatoria porque obligaría al algoritmo a invertir demasiadas iteraciones corrigiendo asignaciones sin sentido antes de poder mejorar de verdad. En su lugar, optamos por una heurística constructiva greedy: para cada tienda, calculamos su distancia a los 10 CEDIS y la asignamos al más cercano que aún tuviera cupo disponible, retornando al final una lista de 10 sublistas donde cada una contiene los índices de las tiendas asignadas a ese centro.

Costo de Ruta

Una de las decisiones más importantes que tomamos fue definir qué significa exactamente que una ruta sea "mejor" que otra. Teníamos dos métricas naturales: la distancia recorrida y el consumo de combustible. Minimizar solo la distancia nos parecía insuficiente porque no captura directamente el impacto económico de la operación, y minimizar solo el combustible tampoco era adecuado de manera aislada. Por eso diseñamos una función que simula el recorrido completo de cada CEDIS, el vehículo parte del centro, visita secuencialmente cada tienda asignada y regresa al origen, acumulando distancia y combustible en cada tramo.

El resultado se expresa mediante una función de costo ponderado que combina ambas métricas, asignándole al combustible un peso del 70% y a la distancia un 30%. Tomamos esta decisión porque el gasto en gasolina es el componente de costo más directo y controlable para la empresa. La distancia sigue siendo relevante como indicador de tiempo de entrega y desgaste vehicular, pero su impacto económico es más indirecto. La función retorna los tres valores por separado, costo ponderado, distancia y gasolina, para poder analizar cada dimensión de forma independiente al momento de presentar resultados.

```
def costo_ruta(ruta, id_cedis, matriz_distancias, matriz_combustible):
costo_distancia =
0 costo_gasolina
= 0 actual =
```

```
id_cedis alpha =
0.3
beta = 0.7

for nodo in ruta:
costo_distancia += matriz_distancias[actual][nodo]
costo_gasolina += matriz_combustible[actual][nodo]
actual = nodo

costo_distancia += matriz_distancias[actual][id_cedis]
costo_gasolina += matriz_combustible[actual][id_cedis]

costo_ponderado = (alpha * costo_distancia) + (beta *
costo_gasolina) return costo_ponderado, costo_distancia,
costo_gasolina
```

Costo Total

Para evaluar qué tan buena es una configuración completa de rutas, necesitábamos una métrica global que resumiera el desempeño de los 10 CEDIS simultáneamente. Construimos una función que aplica el costo de ruta a cada uno de los 10 recorridos y acumula los resultados en tres totales: costo ponderado, distancia y gasolina. El costo ponderado total es el número que el Recocido Simulado intenta reducir en cada iteración, mientras que los otros dos valores nos permiten analizar después cuánto mejoró cada dimensión de manera independiente, lo cual resulta esencial para interpretar y comparar los resultados al final del proceso.

```
def costo_total(rutas, matriz_distancias, matriz_combustible):
total_ponderado = 0
total_distancia = 0
total_gasolina = 0

for id_cedis, ruta in enumerate(rutas):
if ruta:
costo_p, distancia, gasolina = costo_ruta(ruta, id_cedis, matriz_distancias,
matriz_combustible) total_ponderado += costo_p total_distancia += distancia
total_gasolina += gasolina

return total_ponderado, total_distancia,
total_gasolina
```

Generación de Vecinos

Para que el Recocido Simulado pueda explorar el espacio de soluciones, necesita poder generar configuraciones alternativas a partir de la actual. Elegimos el operador de intercambio swap porque es simple, computacionalmente ligero y produce cambios locales y controlados. Primero realizamos una copia profunda de las rutas actuales para no alterar la solución en curso, y luego seleccionamos dos

rutas al azar e intercambiamos una tienda de cada una, generando una solución ligeramente diferente que el algoritmo puede evaluar.

```
def generar_vecino(rutas):
rutas_vecinas = [list(r) for r in
rutas] m = len(rutas_vecinas)


r1 = random.randint(0, m - 1)
posibles = [i for i in range(m) if len(rutas_vecinas[i]) > 0 and i != r1]


if not posibles:
return rutas_vecinas


r2 = random.choice(posibles)


if len(rutas_vecinas[r1]) == 0: nodo =
rutas_vecinas[r2].pop(random.randint(0, len(rutas_vecinas[r2]) - 1))
rutas_vecinas[r1].append(nodo) return rutas_vecinas


idx1 = random.randint(0, len(rutas_vecinas[r1]) - 1)
idx2 = random.randint(0, len(rutas_vecinas[r2]) - 1)


while r1 == r2 and idx1 == idx2:
idx2 = random.randint(0, len(rutas_vecinas[r2]) - 1)


rutas_vecinas[r1][idx1], rutas_vecinas[r2][idx2] = rutas_vecinas[r2][idx2],
rutas_vecinas[r1][idx1] return rutas_vecinas
```

Núcleo del Algoritmo: Recocido Simulado

En cada iteración, generamos un vecino, calculamos su costo y tomamos una decisión: si el vecino es mejor, lo aceptamos de forma incondicional. Si es peor, lo aceptamos de todas formas con una probabilidad que depende de qué tan malo es y de la temperatura actual. Esta es la característica que distingue al Recocido Simulado de un greedy convencional y la razón principal por la que lo elegimos: la capacidad de aceptar soluciones peores de manera controlada le permite escapar de óptimos locales que cualquier método puramente descendente no podría evitar.

Independientemente de lo que se acepte en cada paso, siempre guardamos por separado la mejor solución global encontrada hasta ese momento. Al terminar cada ciclo completo de iteraciones, reducimos la temperatura multiplicándola por el factor de enfriamiento, lo que gradualmente vuelve al algoritmo más selectivo y conservador conforme avanza la búsqueda.

```
def optimizacion_recocido_simulado(df_tiendas, matriz_distancias, matriz_combustible, t0, factor_enfriamiento, iteraciones, tf): actual = solucion_inicial(df_tiendas, matriz_distancias)
actual_costo, actual_distancia, actual_gasolina = costo_total(actual, matriz_distancias, matriz_combustible)

mejor = [list(r) for r in actual]
mejor_costo = actual_costo
mejor_distancia = actual_distancia
mejor_gasolina = actual_gasolina

historial_costos = []
historial_distancias = []
historial_gasolinas = []
historial_mejora = []

t = t0

while t > tf:
for _ in range(iteraciones):
vecino = generar_vecino(actual)
vecino_costo, vecino_distancia, vecino_gasolina = costo_total(vecino, matriz_distancias, matriz_combustible)
delta = vecino_costo - actual_costo

if delta < 0 or random.random() < math.exp(-delta / t):
actual = vecino actual_costo =
vecino_costo actual_distancia
= vecino_distancia
actual_gasolina = vecino_gasolina

if actual_costo < mejor_costo:
mejor = [list(r) for r in actual]
mejor_costo = actual_costo
mejor_distancia =
actual_distancia
mejor_gasolina = actual_gasolina

historial_costos.append(actual_costo)
historial_distancias.append(actual_distancia)
historial_gasolinas.append(actual_gasolina)
historial_mejora.append(
((historial_costos[0] - mejor_costo) / historial_costos[0]) *
100 )

t *= factor_enfriamiento

return mejor, mejor_costo, mejor_distancia, mejor_gasolina, historial_costos, historial_distancias,
historial_gasolinas, historial_mejora
```

Resultados obtenidos:

```
=== SOLUCIÓN INICIAL ===
Costo ponderado: 181.0006
Distancia total: 446.9150
Gasolina total:  67.0373

Ruta 0 → nodos: [53, 69, 81, 91, 93, 96, 98, 99]
  Costo: 24.6098 | Distancia: 60.7650 | Gasolina: 9.1147

Ruta 1 → nodos: [17, 19, 25, 26, 40, 43, 44, 46, 47, 56]
  Costo: 24.2286 | Distancia: 59.8237 | Gasolina: 8.9736

Ruta 2 → nodos: [28, 49, 54, 72, 74, 88, 89, 90, 94, 97]
  Costo: 27.3577 | Distancia: 67.5499 | Gasolina: 10.1325

Ruta 3 → nodos: [21, 33, 36, 45, 50, 79, 82, 84, 92, 95]
  Costo: 15.8008 | Distancia: 39.0144 | Gasolina: 5.8522

Ruta 4 → nodos: [12, 27, 37, 57, 60, 63, 68, 73, 78, 80]
  Costo: 20.3662 | Distancia: 50.2869 | Gasolina: 7.5430

Ruta 5 → nodos: [16, 34]
  Costo: 1.2638 | Distancia: 3.1206 | Gasolina: 0.4681

Ruta 6 → nodos: [31, 35, 38, 52, 58, 59, 67, 70, 75, 77]
  Costo: 14.5618 | Distancia: 35.9551 | Gasolina: 5.3933

Ruta 7 → nodos: [18, 29, 41, 55, 61, 66, 83, 85, 86, 87]
  Costo: 25.0051 | Distancia: 61.7410 | Gasolina: 9.2611

Ruta 8 → nodos: [11, 13, 20, 22, 23, 32, 51, 62, 64, 76]
  Costo: 13.1769 | Distancia: 32.5355 | Gasolina: 4.8803

Ruta 9 → nodos: [10, 14, 15, 24, 30, 39, 42, 48, 65, 71]
  Costo: 14.6299 | Distancia: 36.1231 | Gasolina: 5.4185
```

```
Ruta 0 → nodos: [88, 19, 73, 81, 43, 68, 12, 69]
  Costo: 13.0395 | Distancia: 32.1963 | Gasolina: 4.8294

Ruta 1 → nodos: [46, 40, 96, 63, 26, 91, 17, 25, 99, 56]
  Costo: 9.7344 | Distancia: 24.0355 | Gasolina: 3.6053

Ruta 2 → nodos: [11, 42, 97, 30, 65, 39, 90, 61, 55, 83]
  Costo: 7.7947 | Distancia: 19.2463 | Gasolina: 2.8869

Ruta 3 → nodos: [95, 21, 36, 33, 79, 20, 50, 84, 45, 82]
  Costo: 10.6522 | Distancia: 26.3018 | Gasolina: 3.9453

Ruta 4 → nodos: [47, 44, 59, 93, 37, 57, 27, 98, 60, 78]
  Costo: 12.5444 | Distancia: 30.9737 | Gasolina: 4.6461

Ruta 5 → nodos: [16, 34]
  Costo: 1.2638 | Distancia: 3.1206 | Gasolina: 0.4681

Ruta 6 → nodos: [70, 77, 52, 31, 75, 67, 89, 35, 58, 38]
  Costo: 10.8240 | Distancia: 26.7258 | Gasolina: 4.0089

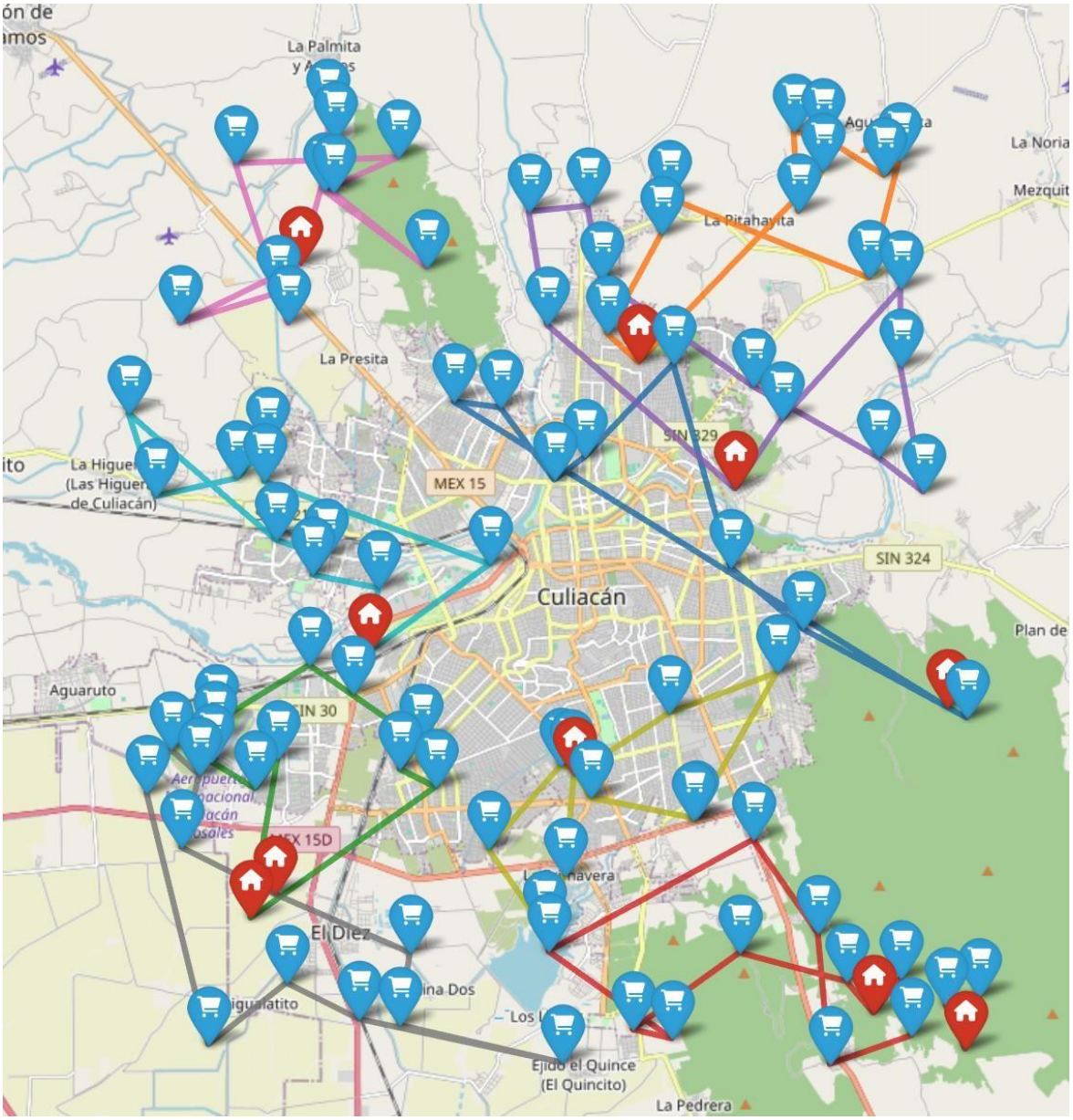
Ruta 7 → nodos: [66, 29, 64, 74, 28, 49, 72, 41, 18, 54]
  Costo: 12.1415 | Distancia: 29.9791 | Gasolina: 4.4969

Ruta 8 → nodos: [80, 53, 92, 22, 32, 23, 51, 62, 76, 13]
  Costo: 8.5692 | Distancia: 21.1586 | Gasolina: 3.1738

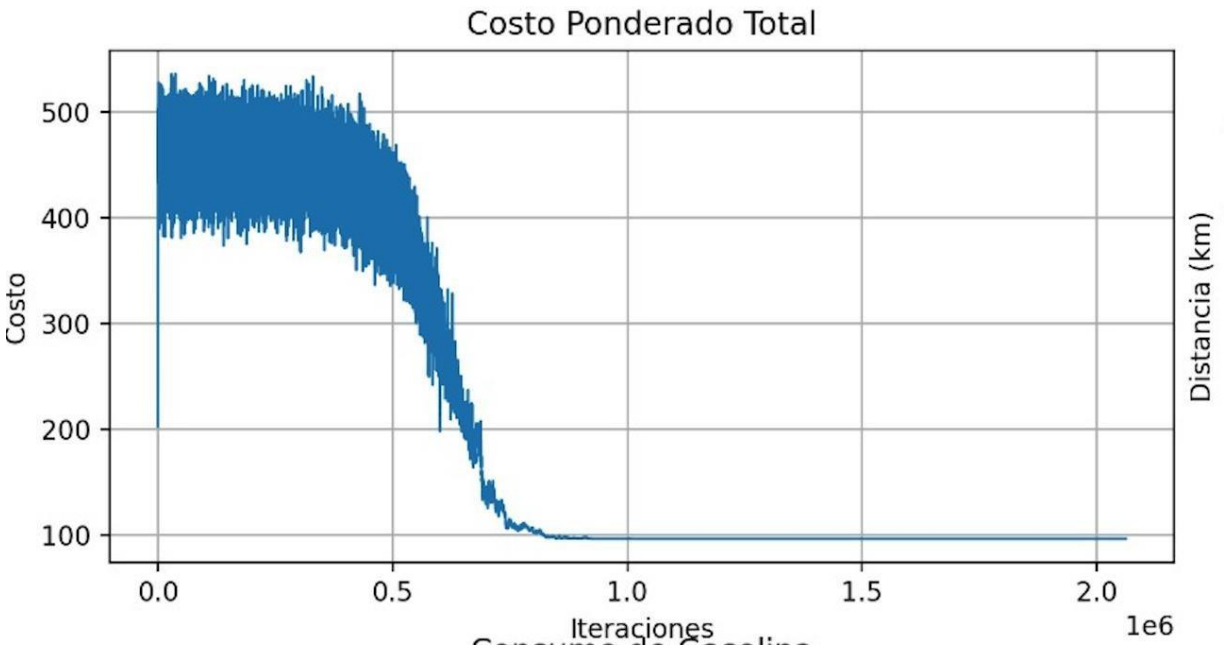
Ruta 9 → nodos: [48, 15, 87, 10, 14, 85, 24, 94, 86, 71]
  Costo: 9.6778 | Distancia: 23.8957 | Gasolina: 3.5844

=== COMPARACIÓN ===
Mejora en costo     → 84.7590 (46.83%)
Mejora en distancia → 209.2816  (46.83%)
Mejora en gasolina  → 31.3922  (46.83%)
```

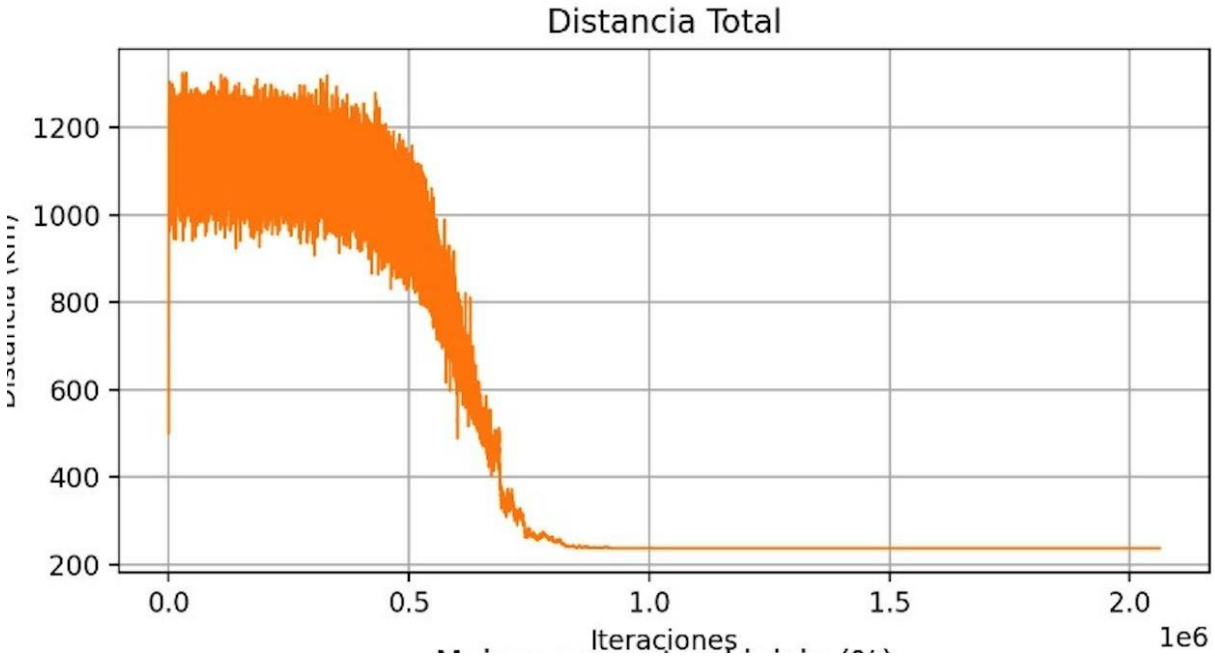
Mapa obtenido:



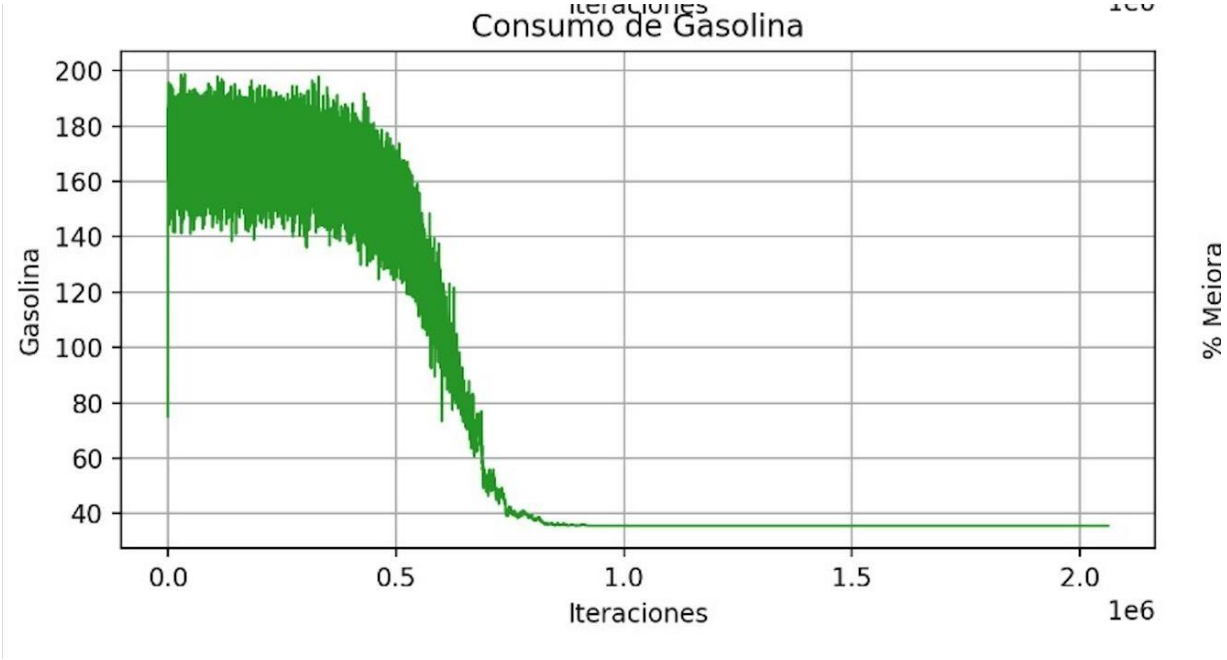
Analisis de efectividad:



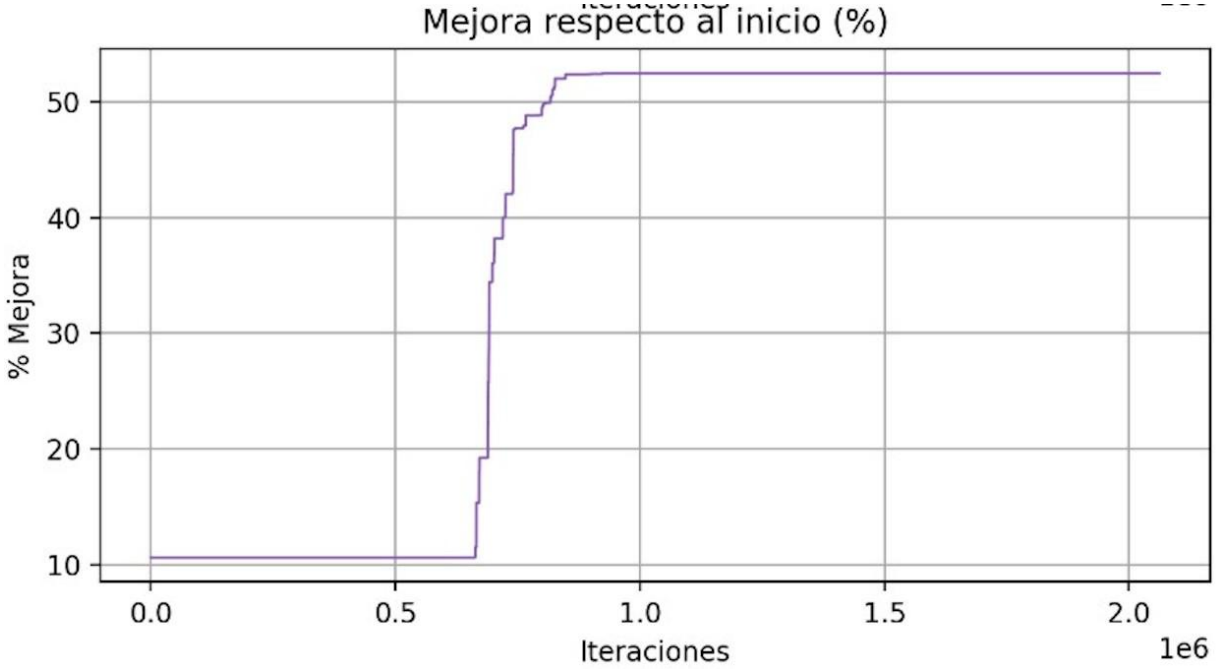
Esta gráfica muestra el rendimiento general del algoritmo. En ella se observa cómo el costo total, que considera tanto la distancia recorrida como el consumo de gasolina, disminuye notablemente a medida que avanzan las iteraciones.



Se puede observar una clara tendencia a la baja, lo que indica que el algoritmo optimizó las rutas para reducir significativamente el consumo total de combustible, logrando estabilizarse en un mínimo.



Esta gráfica muestra la reducción en los kilómetros totales recorridos. Al igual que las otras métricas, la distancia total disminuye a medida que el algoritmo refina las rutas.



Esta gráfica es el resumen del éxito de la optimización. Demuestra que la solución final es casi un 50% mejor que la solución inicial.

8- Conclusiones

El proyecto demostró que el algoritmo de Recocido Simulado constituye una herramienta eficaz para abordar el problema de ruteo de vehículos en una red de distribución real. Partiendo de una solución inicial construida con una heurística equilibrada, el algoritmo logró reducir el costo ponderado total en un 46.83%, pasando de 181.00 a 96.21, con mejoras equivalentes en distancia recorrida y consumo de combustible. Este resultado confirma que la capacidad del algoritmo de aceptar soluciones peores de manera controlada, mediante el criterio probabilístico de Metropolis, le permite escapar de óptimos locales que un método puramente descendente no podría superar.